7. Selection Sort on Random, Reverse and Sorted Arrays

Aim

To write a C program to implement **Selection Sort** and sort random, reverse-sorted, and already-sorted arrays.

Procedure

1. Read or define an array of elements.
2. Divide the array into a sorted and an unsorted portion.
3. Find the smallest element from the unsorted portion.
4. Swap it with the first element of the unsorted portion.
5. Repeat the process until the entire array is sorted.
6. Perform Selection Sort on random, reverse, and sorted arrays.
7. Display the sorted arrays.

main.c

Run ▶

```c
#include <stdio.h>
void selectionSort(int arr[], int n)
{
    int i, j, min, temp;
    for (i = 0; i < n - 1; i++)
    {
        min = i;
        for (j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[min])
            {
                min = j;
            }
        }
        temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }
}
void display(int arr[], int n)
```

Output

```
Random Array Before Sorting: 64 25 12 22 11
Random Array After Sorting:  11 12 22 25 64

Reverse Array Before Sorting: 50 40 30 20 10
Reverse Array After Sorting:  10 20 30 40 50

Sorted Array Before Sorting: 10 20 30 40 50
Sorted Array After Sorting:  10 20 30 40 50
```

main.c

```c
{
    int i;
    for (i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
int main()
{
    int random[] = {64, 25, 12, 22, 11};
    int reverse[] = {50, 40, 30, 20, 10};
    int sorted[] = {10, 20, 30, 40, 50};
    int n = 5;
    printf("Random Array Before Sorting: ");
    display(random, n);
    selectionSort(random, n);
    printf("Random Array After Sorting:  ");
    display(random, n);
    printf("\nReverse Array Before Sorting: ");
    display(reverse, n);
    selectionSort(reverse, n);
```

```c
    int reverse[] = {50, 40, 30, 20, 10};
    int sorted[] = {10, 20, 30, 40, 50};
    int n = 5;
    printf("Random Array Before Sorting: ");
    display(random, n);
    selectionSort(random, n);
    printf("Random Array After Sorting:  ");
    display(random, n);
    printf("\nReverse Array Before Sorting: ");
    display(reverse, n);
    selectionSort(reverse, n);
    printf("Reverse Array After Sorting:  ");
    display(reverse, n);
    printf("\nSorted Array Before Sorting: ");
    display(sorted, n);
    selectionSort(sorted, n);
    printf("Sorted Array After Sorting:  ");
    display(sorted, n);
    return 0;
}
```

Result

Thus, the C program successfully sorts **random, reverse-sorted, and already-sorted arrays** using the Selection Sort algorithm.

Time Complexity: $O(n^2)$ for best, average, and worst cases.